How your Boat Rental site works

A practical walkthrough of the current files, buttons, saved records, and reports.

FILE	PURPOSE
index.php	Login screen. JavaScript checks the demo username and password.
main.php	Admin and staff screen layout, navigation, buttons, forms, and report controls.
main.js	Click and submit handlers, validation, localStorage, calculations, filters, and screen rendering.
styles.css	Colors, spacing, tables, boat cards, responsive layout, and print layout.
database/it37project.sql	A schema prepared for later database connection. The current pages do not use it.

The main flow

```
Click or submit in main.php  ->  main.js event handler  ->  validate or filter data
                  ->  writeRecords() saves JSON to browser localStorage
                  ->  renderPortal() rebuilds tables, cards, totals, and report output
```

The pages use .php filenames so XAMPP can serve them, but the current interface is HTML and JavaScript. PHP does not yet query MySQL or store the records. The admin and staff views share browser records in the same browser profile and localhost origin.

This guide describes the current code in main.js, main.php, and styles.css, including the Add/Edit/Remove actions and the report filter path.

Login, navigation, and shared forms

Login is a front-end demonstration. The login submit handler compares fixed values and redirects to main.php with a role in the URL. The role query controls which interface details appear; it is not verified by PHP or protected with a session.

```
admin123 / admin123  ->  main.php?role=admin
staff123 / staff123   ->  main.php?role=staff
```

The sidebar uses URL hashes such as #resources. showScreen() reads the hash, shows the matching section, hides the other sections, updates the title and active navigation item, and scrolls to the top.

How Add and Edit reach the shared Save function

```
Add button has data-form="clients"
click listener -> openRecordForm('clients')
Edit row button -> openRecordForm('clients', client.id)
form submit -> savePortalRecord()
```

Every Add/New button uses a data-form value. The click listener passes that value to openRecordForm(kind). The form layout selected by kind determines which fields are built. For an Edit action, the table passes the saved record ID as editId, so the form fields are filled with the existing values.

Save reads the kind and editId stored on the dialog. A blank editId means create a new record; a populated editId means find and replace that record while keeping its ID. The submit handler prevents a page reload and routes supported records to savePortalRecord().

Closing the dialog with X or Cancel discards unsaved form input. Cancel booking is a separate action that changes an already-saved booking.

Button map: Add, Edit, Remove, and Cancel

The action labels below point to the actual JavaScript path. Buttons are created in main.php or as table rows by renderPortal(); main.js attaches the click behavior.

SCREEN / BUTTON	HANDLER PATH	WHAT IT DOES
Resources / Add resource	data-form=resources -> openRecordForm('resources') -> savePortalRecord() resources branch	Validates boat name, type, capacity, rate, status, and optional picture; pushes a new resource.
Resources / Edit	renderPortal() row action -> openRecordForm('resources', resource.id) -> resources save branch	Prefills the selected boat; save replaces its object and keeps its ID and photo unless a new photo is selected.
Resources / Remove	renderPortal() row action -> removeManagedRecord('resources', id)	Asks for confirmation. Blocks removal while a non-cancelled booking has not ended; otherwise removes the resource.
Clients / Add client	data-form=clients -> openRecordForm('clients') -> savePortalRecord() clients branch	Validates client fields and appends a client with a new ID.
Clients / Edit	renderPortal() row action -> openRecordForm('clients', client.id) -> clients save branch	Loads that client's details, then replaces the same record on Save.
Clients / Remove	renderPortal() row action -> removeManagedRecord('clients', id)	Confirms and removes the client record. Older bookings keep the saved client name snapshot.
Staff / Add staff	data-form=staff -> openRecordForm('staff') -> savePortalRecord() staff branch	Validates staff details and appends a staff member.
Staff / Edit	renderPortal() row action -> openRecordForm('staff', member.id) -> staff save branch	Loads that member's fields and replaces their record on Save.
Staff / Remove	renderPortal() row action -> removeManagedRecord('staff', id)	Confirms; blocks removal if that member has a non-cancelled booking that has not ended.
Payments / Record payment	data-form=payments -> openRecordForm('payments') -> savePortalRecord() payments branch	Adds a receipt linked to a booking. It does not change the booking price or process an actual charge.
Bookings / New booking	data-form=bookings -> openRecordForm('bookings') -> savePortalRecord() bookings branch	Creates a booking from registered client, available boat, date range, guest count, and available staff.
Bookings / Cancel booking	renderPortal() row action -> cancelBooking(id) -> cancel-form submit handler	Records cancellation status, who cancelled, their role, and time. The booking remains in history.

Resource removal checks booking.resourceId against the selected resource ID, ignores cancelled bookings, and requires the booking end time to be later than now. That includes a booking currently in progress. Past and cancelled bookings do not block removal.

Daily resource rates and booking totals

The Resources rate is now a price per rental day. New bookings calculate the charge from the selected boat's daily price and the rental length.

How a rental day is counted

The booking start and return are still date-and-time values. The code divides the elapsed time by 24 hours and rounds up. Any started 24-hour block counts as a full day, and a booking always costs at least one day.

```
days = max(1, ceil((return time - departure time) / 24 hours))
total = daily rate x billable days
```

Example: a boat priced at PHP 5,000 per day rented for 30 hours is charged for 2 days, for a total of PHP 10,000.

Where the code is

Resource price label	main.js formLayouts and resource renderers	Shows the daily rate in the Add/Edit form, boat picker, cards, and Resources table.
Day calculation	main.js billableRentalDays()	Counts each started 24-hour block as one charged day.
Booking total	main.js savePortalRecord(), bookings branch	Saves dailyRate, days, and total on each new booking.
Database schema	database/it37project.sql	Uses pricePerDay for boats and dailyRateAtBooking for bookings.

Saved records

Existing browser records are kept. Resource rate numbers are not changed, so they now mean PHP per day; review the amounts on the Resources page and edit them to the intended daily prices. Existing booking totals stay as originally saved. New bookings use the daily calculation.

For an earlier database created from the hourly-rate schema, database/upgrade-daily-rates.sql renames the rate columns and preserves their numbers and saved totals. Back up the database before using that migration, then review boat prices.

Reports: what happens when filters are pressed

The report form is in main.php. Its controls have IDs report-from, report-to, report-resource, report-client, and report-payment-status. The Generate report button submits this form. The behavior is wired in main.js near the reportForm listener.

STEP	CODE / EVENT	RESULT
1. Change a filter	reportForm input listener	Hides the previous report and disables Print. The changed filters are not treated as applied yet.
2. Press Generate report	reportForm submit listener	Prevents normal form reload. Reads the dates and checks that both exist and From is not after To.
3. Save selections for this view	report-period dataset	Stores dates, resource ID, client ID, payment status, and visible labels temporarily on the page.
4. Refresh data	renderPortal()	Re-reads records, refreshes resource/client dropdown options, then calls renderReport(records).
5. Filter and calculate	renderReport(records)	Keeps matching bookings, calculates booking count and money totals, and rebuilds report rows.
6. Print	print-button click listener	Calls window.print() only when a generated report is visible. Browser print settings can save as PDF.

What each filter means

Dates filter by booking departure date and include both endpoints. Resource filters by resource ID; Client filters by client ID (or a name key for older records). Payment status filters by the derived booking payment state. Changing any filter hides the previous result until Generate report is pressed again.

Payment status choices

CHOICE	MEANING
All payment statuses	Keep every payment state.
Paid	The remaining balance is zero.
Partially paid	At least one payment exists and a balance remains.
Unpaid	No payment has been recorded and a balance remains.
Cancelled bookings	Booking status is Cancelled; it remains in report history.

The page calls renderPortal() after a valid submit. That function also redraws other screens from the same records, then the report renderer reads the submitted report selections from the report-period element.

Report totals, payments, and cancellation

For the bookings that remain after filtering, renderReport() displays each booking and derives the totals. Amounts are summed in integer cents before formatting as PHP currency to avoid floating-point rounding drift.

REPORT VALUE	HOW IT IS CALCULATED
Bookings	Number of matching booking rows, including cancelled rows when selected.
Booked value	Sum of non-cancelled booking totals.
Total paid	Receipts linked to matching bookings; saved receipts remain visible even for cancelled bookings.
Outstanding	Remaining balances for matching non-cancelled bookings. Cancelled bookings contribute zero.

Payment status comes from the booking's price and its saved receipts. No receipt plus a balance is Unpaid; receipts with a balance are Partial; a zero balance is Paid. Cancelled booking status takes precedence. Recording a payment adds a receipt; it does not contact a bank, card network, or payment provider.

Cancellation path

```
Cancel booking button -> cancelBooking(id) -> choose active staff or admin
Confirm cancellation -> validate selection -> set status and canceller fields
                  -> writeRecords(records) -> renderPortal() redraws tables and reports
```

A cancellation stores the selected person's name, role, staff ID when relevant, and timestamp. It does not delete the booking or automatically create a refund. Cancelled bookings stop blocking boat and staff time. Existing receipts remain recorded.

Report date meaning

The period refers to rental departure dates, not the dates when payments were received. A receipt entered this month for a rental departing next month appears in a report whose dates include that departure. The report is a filtered view, not a permanent saved report file.

Other screens and useful function map

Dashboard cards and tables are calculated from the same saved lists. They are rebuilt by renderPortal() after saves, cancellations, and relevant filter or navigation events.

SCREEN / VALUE	FUNCTION OR RULE
Dashboard	Counts resources, currently available boats, active bookings, and bookings with a remaining balance.
Availability	Date controls call updateSchedule(); renderSchedule() draws Morning, Afternoon, and Evening blocks for each resource.
Resources	renderPortal() rebuilds boat rows. Staff boat cards use renderBoatCatalog().
Payments	renderPayments() groups each booking with its linked receipts, paid total, balance, and derived status.
Reports	updateReportFilters() fills Resource/Client choices; renderReport() applies submitted filters and totals.
Search/status filters	matchesFilters() checks the section's current text and status controls; matching rows are redrawn.

Reading the action comments in main.js

COMMENT LABEL	FOLLOW IT TO
RESOURCES / EDIT or REMOVE	The resource row action in renderPortal(), then openRecordForm() or removeManagedRecord().
CLIENTS / EDIT or REMOVE	The client row action, then the clients save branch or shared removal function.
STAFF / EDIT or REMOVE	The staff row action, then the staff save branch or shared removal function.
BOOKINGS / CANCEL	cancelBooking(), then the cancellation form submit listener.
FILTER CHANGE / GENERATE REPORT / PRINT REPORT	The matching report form or print button listener near reportForm.
REPORT FILTER OPTIONS / REPORT RESULTS	updateReportFilters() and renderReport(), called by renderPortal().

The comments are signposts: the click handler starts the action, the record function does the checks and update, writeRecords() saves, and renderPortal() refreshes the display.

Files, roles, and current limits

What the CSS handles

styles.css controls visual presentation: colors, type, spacing, sidebar and screen layout, form dialogs, tables, status badges, boat cards, responsive behavior, and print rules. CSS does not save data or handle button clicks. main.js supplies classes and content; CSS decides how those elements look.

Admin and staff differences

The role is read from the URL query. Admin can add, edit, and remove resources. Staff can browse boats and start bookings by choosing a boat card; resource management actions are hidden and the resource save handler also checks the role. Both views use the same localStorage records. Staff records in the Staff page do not create login accounts.

Where records are saved

The current key is boatRental.sharedRecords.v1 in browser localStorage. Refreshing normally preserves records in the same browser profile and origin. Another browser, computer, host, or port has a separate store. The SQL schema in database/ is not connected to these screens yet.

If something appears missing

Check the screen's search/status filters first. For reports, confirm the departure date range and Resource, Client, and Payment status choices, then press Generate report. For a booking or removal rule, check the booking's saved resource ID, status, and end time. Do not clear browser site data while looking for saved local records.

Current limits

The login is a fixed front-end demonstration, not server-side authentication. Records use localStorage, not MySQL. The account form is visual only. Payment recording does not process a charge, cancellation does not issue a refund, and bookings are not automatically marked Completed after their end time.

The core pattern

```
HTML button/form -> main.js event listener -> handler and validation
-> records object -> writeRecords() -> localStorage -> renderPortal()
Report controls -> submit handler -> report-period dataset -> renderReport()
```

Updated 13 October 2026 to describe the current Add/Edit/Remove actions and report filter handlers.